

浙江大学

本科实验报告

课程名称:	课程综合实践 I- 深度学习基础——理论与实践
姓 名:	李昊泽
学 院:	计算机科学与技术学院
系:	
专 业:	计算机科学与技术
学 号:	3250102780
指导教师:	王总辉

2026 年 8 月 31 日

浙江大学实验报告

课程名称: 课程综合实践 I- 深度学习基础——理论与实践 实验类型: 综合

实验项目名称: 实验一

学生姓名: 李昊泽 专业: 计算机科学与技术 学号: 3250102780

实验地点: 紫金港海洋大楼 11 机房 实验日期: 2026 年 8 月 31 日 指导老师: 王总辉

摘要

本实验由两个相互补充的任务组成。第一部分在 Kaggle 平台上完成 PM2.5 浓度预测: 将连续 9 小时的 18 项空气质量指标整理为特征, 使用线性回归和 Adagrad 进行训练, 并生成符合竞赛格式的 `submit.csv`。第二部分在 LMCC 平台上完成 MNIST 手写数字识别: 把灰度图像转换为张量, 利用全连接神经网络、交叉熵损失和 Adam 优化器完成十分类训练。

两个任务分别对应回归与分类。它们使用的模型并不复杂, 但完整经历了数据读取、预处理、特征表示、模型训练、指标观察和结果提交。

实验目的和要求

1. 熟悉 Kaggle Notebook、LMCC 和 Jupyter Notebook 的基本工作方式, 理解在线实验环境中输入目录、工作目录和计算设备的区别。
2. 完成 PM2.5 回归预测, 掌握时间序列滑动窗口、特征标准化、线性回归和 Adagrad 的基本实现。
3. 完成 MNIST 手写数字分类, 掌握图像张量化、批量加载、前向传播、反向传播和验证的基本流程。
4. 比较回归与分类任务在输入表示、损失函数、输出形式和评价指标上的差异, 并反思实验结果能够说明什么、不能说明什么。

实验环境

项目	环境或工具	用途
PM2.5 预测	Kaggle Notebook、NumPy、Pandas	读取竞赛数据、训练线性回归、生成提交文件
手写数字识别	LMCC、PyTorch、TorchVision、CUDA	加载 MNIST、建立神经网络并进行训练与验证

实验一：Kaggle 平台 PM2.5 预测

一、任务

要求根据过去一段时间的空气质量观测，预测下一小时的 PM2.5 浓度。目标值是连续数值，因此它属于回归问题，而不是分类问题。训练数据按“日期 - 测站 - 测项 - 24 小时观测值”的形式保存，每一天包含 18 项指标。实际读取结果为 3240 行、27 列，对应 12 个月、每月 15 天、每天 18 项指标。

```
1 Kaggle Input 中的文件:
2 /kaggle/input/competitions/2026zjutest1/test.csv
3 /kaggle/input/competitions/2026zjutest1/train.csv
4 使用的数据目录: /kaggle/input/competitions/2026zjutest1
5 是否提供样例提交文件: False
```

二、数据清洗与编码处理

训练集前三列保存描述信息，真正参与计算的是后面的 24 个小时。代码把 NR 和无法转换为数值的内容统一处理为 0，再转成 NumPy 浮点数组。PM2.5 在每组 18 项指标中的索引由列名自动查找，当前数据得到的索引为 9。

```
1 train_num = (
2     train_df.iloc[:, 3:]
3     .replace("NR", 0)
4     .apply(pd.to_numeric, errors="coerce")
5     .to_numpy(dtype=float)
6 )
```

实际操作中曾出现中文表头乱码。最初使用 ISO-8859-1 虽然不会触发解码异常，却会把中文按错误字符显示；改用 gb18030 后表头能够正确读取。

三、构造 9 小时滑动窗口

设窗口长度为 9。对每个月的连续 360 小时，从第 0 小时开始依次截取 9 小时数据，并把紧接着的下一小时 PM2.5 作为标签。一个输入窗口的形状为 18×9 ，展平后得到 162 维特征：

$$x_t = [z_t, z_{t+1}, \dots, z_{t+8}], \quad y_t = \text{PM2.5}_{t+9}$$

窗口每次向后移动 1 小时，因此每个月可得到 $360 - 9 = 351$ 个样本，12 个月共得到：

$$12 \times 351 = 4212$$

Notebook 的实际输出为 `x.shape=(4212, 162)`、`y.shape=(4212, 1)`，与推导一致。这里没有让窗口跨越月份边界，是因为相邻月份的数据在原始数据集中并不一定连续；如果强行连接，可能构造出并不存在的时间关系。

```
1 feature = month_data[month, :, start:start +  
9].reshape(-1)  
2 target = month_data[month, PM25_INDEX, start + 9]  
3 x_list.append(feature)  
4 y_list.append(target)
```

Python

训练特征形状：(4212, 162)

训练标签形状：(4212, 1)

课件数据的预期形状：(4212, 162) 和 (4212, 1)

四、标准化与线性回归

162 个特征来自不同指标和不同时间位置，量纲差异较大。若直接训练，数值范围较大的指标可能主导梯度，因此按列进行标准化：

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}$$

均值 μ_j 和标准差 σ_j 只使用训练集计算。测试集必须复用同一组统计量，否则训练阶段和预测阶段会处于不同的坐标尺度，也会引入不必要的信息泄漏。

在特征前增加常数 1 后，模型可写为：

$$\hat{y} = w^T x + b$$

训练过程中使用 RMSE 观察误差：

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2}$$

代码计算的梯度对应均方误差 MSE，而日志以 RMSE 的形式展示。两者的最优点相同，因此不会改变最终要寻找的最优权重，但梯度尺度不同。优化器使用 Adagrad，累计各参数历史梯度的平方，并据此缩小频繁出现大梯度参数的有效学习率。

```
1 error = x_train @ w - y  
2 gradient = 2 * x_train.T @ error / len(x_train)  
3 adagrad += gradient ** 2  
4 w -= learning_rate * gradient / np.sqrt(adagrad + eps)
```

Python

五、训练结果与分析

模型共训练 1000 次，每 100 次打印一次训练集 RMSE。保存的运行记录如下：

迭代次数	0	300	600	900
训练 RMSE	26.1037	12.6051	8.5829	7.8357

第 100 次的 RMSE 一度上升到 35.1418，之后才持续下降。这说明基础学习率 100 在训练初期较激进，而 Adagrad 随着梯度平方的累积逐渐降低有效步长，训练才转入稳定区间。从 300 次以后，下降速度明显变慢，继续增加迭代次数仍可能改善训练误差，但收益已经减小。

需要强调的是，这些数值只代表训练集拟合情况。当前实现没有单独划分验证集，所以不能仅凭 7.8357 判断模型对未知数据的效果。Kaggle 的公开分数更接近外部验证，但它也只是特定测试集和评价规则下的结果。更严谨的做法是按时间划分验证集，例如使用较后的月份验证，避免随机打乱导致未来信息进入训练集。

1	0: RMSE=26.103713	terminal
2	100: RMSE=35.141762	
3	200: RMSE=18.377660	
4	300: RMSE=12.605111	
5	400: RMSE=10.326857	
6	500: RMSE=9.210694	
7	600: RMSE=8.582850	
8	700: RMSE=8.208529	
9	800: RMSE=7.979155	
10	900: RMSE=7.835748	
11	权重已保存: /kaggle/working/weight.npy	

六、测试集预测

测试集共有 1080 行，每 18 行构成一个样本，每行包含连续 9 小时的观测，因此共得到 60 个测试样本。测试特征按照与训练集完全相同的顺序展平、标准化并添加偏置项，再与训练得到的权重相乘。

前 5 条预测值为 23.7778、23.6731、81.4144、60.9326 和 27.5764。最终生成的 `submit.csv` 包含 `id`、`value` 两列，共 60 行数据；本地检查确认 ID 范围为 1 至 60，预测值均为有限数且不存在缺失值。

原始测试集形状: (1080, 11)

	0	1	2	3	4	5	6	7	8	9	10
0	id_0	AMB_TEMP	20	20	19	18	16	15	14	14	13
1	id_0	CH4	1.8	1.8	1.8	1.8	1.8	1.8	1.8	1.8	1.8
2	id_0	CO	0.27	0.26	0.3	0.48	0.49	0.42	0.36	0.32	0.28
3	id_0	NMHC	0.15	0.15	0.18	0.27	0.25	0.18	0.13	0.1	0.08
4	id_0	NO	1.6	1.7	1.4	1.3	1.2	1.2	0.9	0.9	1.1

预测数量: 60

前5条预测: [23.77782701 23.67305436 81.41437471 60.93256853 27.57638246]

七、讨论与总结

PM2.5 实验表面上是在训练一个线性模型，实际难点更多集中在数据结构上：先恢复时间顺序，再定义什么是一个样本。线性回归的表达有限，却提供了一个容易解释的基准。

实验二：LMCC 平台手写数字识别

一、任务与数据集

MNIST 包含 70000 张手写数字灰度图像，其中 60000 张用于训练，10000 张作为测试数据。每张图像大小为 28×28 ，只有一个灰度通道，标签为 0 至 9。与 PM2.5 的连续数值预测不同，这里需要从十个类别中选择一个，因此属于多分类任务。

二、图像张量化与批量加载

原始图像以 PIL 图像形式读取。`ToTensor` 将其转换为 `C x H x W` 排列的浮点张量，形状为 `(1, 28, 28)`，并把像素值由 0 至 255 映射到 0.0 至 1.0。相比直接使用整数像素，统一的浮点范围更适合梯度优化。

数据通过 `DataLoader` 按批次送入模型，批大小设置为 32，训练集启用随机打乱，验证集不打乱。批量训练在计算效率和梯度波动之间取得折中，也避免一次把全部图像送入显存。

```
1 trans = transforms.Compose([transforms.ToTensor()])
2 train_loader = DataLoader(train_set, batch_size=32, shuffle=True)
3 valid_loader = DataLoader(valid_set, batch_size=32)
```

三、神经网络结构

本实验使用一个基础全连接神经网络：

层	输入与输出	作用
Flatten	<code>1x28x28 → 784</code>	将二维图像展开为一维向量
Linear + ReLU	<code>784 → 512</code>	学习像素组合与数字形状之间的非线性关系
Linear + ReLU	<code>512 → 512</code>	进一步组合中间特征
Linear	<code>512 → 10</code>	输出十个类别的 logits

输出层不直接使用 Softmax，因为 `CrossEntropyLoss` 会在内部完成适合数值稳定性的相关计算。预测时取十个输出中最大值的索引作为数字类别。

```
1 model = nn.Sequential(
2     nn.Flatten(),
3     nn.Linear(28 * 28, 512), nn.ReLU(),
4     nn.Linear(512, 512), nn.ReLU(),
5     nn.Linear(512, 10),
6 ).to(device)
```

```
model.to(device)

[42] ✓ 0.0s Python

... Sequential(
  (0): Flatten(start_dim=1, end_dim=-1)
  (1): Linear(in_features=784, out_features=512, bias=True)
  (2): ReLU()
  (3): Linear(in_features=512, out_features=512, bias=True)
  (4): ReLU()
  (5): Linear(in_features=512, out_features=10, bias=True)
)
```

四、损失函数、优化器与训练循环

交叉熵损失衡量预测分布与真实类别之间的差异，其基本形式为：

$$L = - \sum_{k=1}^{10} y_k \log p_k$$

优化器选择 Adam。每个训练批次依次执行：数据移动到 GPU、前向传播、清空梯度、计算损失、反向传播和更新参数。验证阶段则切换到 `model.eval()`，并在 `torch.no_grad()` 中运行，避免记录梯度和意外更新参数。

Notebook 计划训练 5 个 epoch。每个 epoch 都完整遍历一次训练集，并在验证集上计算损失与准确率。训练准确率反映拟合程度，验证准确率则更能说明模型面对未参与参数更新的数据时是否仍然有效。下面这几行是单个训练批次中的关键更新步骤：

```
1 optimizer.zero_grad()
2 output = model(x)
3 batch_loss = loss_function(output, y)
4 batch_loss.backward()
5 optimizer.step()
```



```
... Epoch: 0
Train - Loss: 384.9690 Accuracy: 0.9375
Valid - Loss: 34.5678 Accuracy: 0.9675
Epoch: 1
Train - Loss: 157.4247 Accuracy: 0.9739
Valid - Loss: 26.4496 Accuracy: 0.9744
Epoch: 2
Train - Loss: 109.1010 Accuracy: 0.9816
Valid - Loss: 25.3899 Accuracy: 0.9772
Epoch: 3
Train - Loss: 86.7424 Accuracy: 0.9855
Valid - Loss: 20.9125 Accuracy: 0.9794
Epoch: 4
Train - Loss: 66.8986 Accuracy: 0.9888
Valid - Loss: 24.2270 Accuracy: 0.9780

我们已经接近 100% 了！让我们测试原始样本来验证这一点。可以像使用函数一
样使用我们的模型：

prediction = model(x_0_device)
prediction
52] ✓ 0.3s Python
tensor([[ -25.0990,  -4.9936, -15.1744,   7.1018, -28.0515,  16.0384, -
        -13.6462,  -9.8901,  -7.7673]], device='mps:0',
        grad_fn=<LinearBackward0>)
```

Figure 1: 五轮训练与验证结果

最后一轮训练准确率为 98.88%，验证准确率为 97.80%；训练损失为 66.8986，验证损失为 24.2270。

五、实验结果分析

仅总体准确率仍不够完整。可以进一步输出混淆矩阵，观察哪些数字容易混淆，例如书写接近的 3、5、8；再展示若干错误样本，判断问题来自书写歧义、图像质量，还是模型能力不足。相比单纯追求更高的百分比，这类分析更能解释模型到底学到了什么。

对于混淆矩阵的应用，本报告不详细展开，给出几个参考阅读：

- [scikit-learn: 使用混淆矩阵评价分类器](#) 该页面介绍了混淆矩阵的基本含义及其可视化方法。矩阵对角线表示正确分类的样本，非对角线表示误分类样本；通过归一化还可以比较不同类别的识别效果。
- [PyTorch 官方 MNIST 示例](#) 该项目提供了基于卷积神经网络的 MNIST 分类示例，可以与本实验使用的全连接神经网络进行比较，进一步研究不同模型结构对识别效果的影响。

六、本部分小结

MNIST 实验把“模型如何学习”展示得比线性回归更直观：损失函数给出错误程度，反向传播把错误分配给各层参数，优化器再依据梯度调整权重。需要理解的重点不是记住 API，而是区分训练模式与验证模式，并说明评价为什么必须使用未参与参数更新的数据。

参考资料

1. 《实验一》课程课件，课程实验资料。
2. `lab1-LMCC.ipynb`，LMCC MNIST 手写数字识别实验代码。
3. 本报告的部分文字和代码修改建议由 AI 辅助生成。所有相关内容均经过本人审阅、修改和实际运行验证，本人已充分理解其原理与实现，并对报告的最终内容负责。